

Unreal Engine Bases et architecture

Table des matières

1	Introduction	1
2	Fondations et architecture	1
2.1	Gameplay Framework	1
3	Subsystems et architecture orientée data	1
3.1	Subsystems	1
3.2	Data oriented : définition	1
4	Modules et Build System	2
4.1	Modules	2
4.2	Build System	2
5	Reflection System, GC Blueprint Exposure	3
5.1	UObject vs AActor	3
5.2	UProperty	3
5.3	TObjectPtr	4
5.4	Fonctionnement du garbage collector	4

1 Introduction

Ce fichier documentera les 6 mois de travail sur un programme de montée en compétences sur le moteur Unreal Engine.

2 Fondations et architecture

Le gameplay framework est une collection de classe interconnectée modulaire sur lesquelles on peut baser n'importe quel projet unreal engine.

2.1 Gameplay Framework

La première d'entre elles à entrer en jeu et le Game Instance qui est lancé au même moment que le moteur et se ferme au même moment également, dans le cadre du développement d'un jeu elle est principalement utile pour stocker les données qui transitent entre les niveaux et les systèmes persistants (sauvegarde, fonctionnalités online, etc..) étant donné que c'est la seule classe à survivre systématiquement aux changements de niveaux, tandis que les autres sont recrées ou simplement détruites.

Ensuite après le chargement du niveau c'est le GameMode qui est directement instancié, il définit les règles et les acteurs à utiliser et spawn à chaque parties, c'est en quelque sorte le gestionnaire de sa session de jeu. En multijoueur il n'existe que du point de vue du serveur car il détient l'autorité sur l'ensemble des règles du jeu.

Quant au game state et player state ils gèrent pour le premier l'état de la partie avec les informations pertinentes qui la caractérise ainsi qu'une liste des joueurs présents dans celle-ci, chacun géré par leur propre player state créée lorsqu'ils rejoignent la partie. Le game state est répliqué à tous les joueurs d'une partie et le player state contient des infos propres à chacun d'entre eux tel que son score ou son équipe.

En parlant du player il est composé de deux composantes principales :

- Le controller qui détient les trigger d'inputs permettant de déclencher des actions
- Le pawn qui est la manifestation physique du joueur contrôlé par le controller, on utilisera souvent sa classe dérivée character permettant de créer un joueur fonctionnel avec un component character movement très utile notamment.

3 Subsystems et architecture orientée data

3.1 Subsystems

Les subsystems sont des sortes de singletons héritant du cycle de vie de leurs classes respectives, elles permettent d'éviter de surcharger les classes de bases d'Unreal ou de les remplacer. Ils permettent de construire des systèmes bien séparés, faciles d'accès et gérés de manière automatique.

3.2 Data oriented : définition

L'architecture orientée data consiste en la séparation stricte des données et du système, on dit que le code consomme les données, dans le sens où il les parcourt pour décrire leurs comportements.

Souvent utilisé pour des raisons de performances, elle permet de limiter la création d'objets inutiles. Elle permet aussi et surtout une flexibilité des systèmes notamment pour les inventaires, différents types d'ennemis etc.

Dans Unreal Engine on utilisera les data assets, équivalents conceptuels des scriptables objects de Unity, permettant de stocker des types de données précis. L'héritage s'applique également au DA offrant une flexibilité très élevée.

4 Modules et Build System

4.1 Modules

Les modules permettent d'encapsuler des fonctionnalités cohérentes (gameplay, outils, systèmes techniques) au sein d'un projet Unreal Engine. Par défaut, un projet Unreal ne contient qu'un seul module principal, généralement nommé comme le projet lui-même.

La création de modules supplémentaires permet de structurer l'architecture du projet de manière plus modulaire, en séparant clairement les responsabilités et en maîtrisant explicitement les dépendances entre systèmes. Chaque module constitue une unité de compilation logique, ce qui permet à Unreal Build Tool de ne recompiler que les modules impactés par une modification, réduisant ainsi significativement les temps de build dans les projets de grande ampleur.

Les modules représentent également la base du système de plugins d'Unreal Engine : un plugin n'est, fondamentalement, qu'un ou plusieurs modules rendus indépendants du projet hôte. Cette indépendance facilite la réutilisation de fonctionnalités entre projets et encourage une architecture plus propre et maintenable.

Il convient toutefois de ne pas sur-segmenter un projet en modules. Certaines fonctionnalités fortement couplées au gameplay spécifique du projet ou à ses données principales ont davantage leur place dans le module principal, afin d'éviter une complexité inutile et une multiplication des dépendances.

Chaque module possède son propre fichier `.Build.cs`, qui définit :

- ses dépendances publiques et privées,
- les règles de compilation spécifiques à la plateforme ou à la Target.

La distinction entre dépendances publiques et privées est essentielle :

- Une dépendance publique est propagée aux modules dépendants.
- Une dépendance privée est utilisée uniquement dans l'implémentation interne du module.

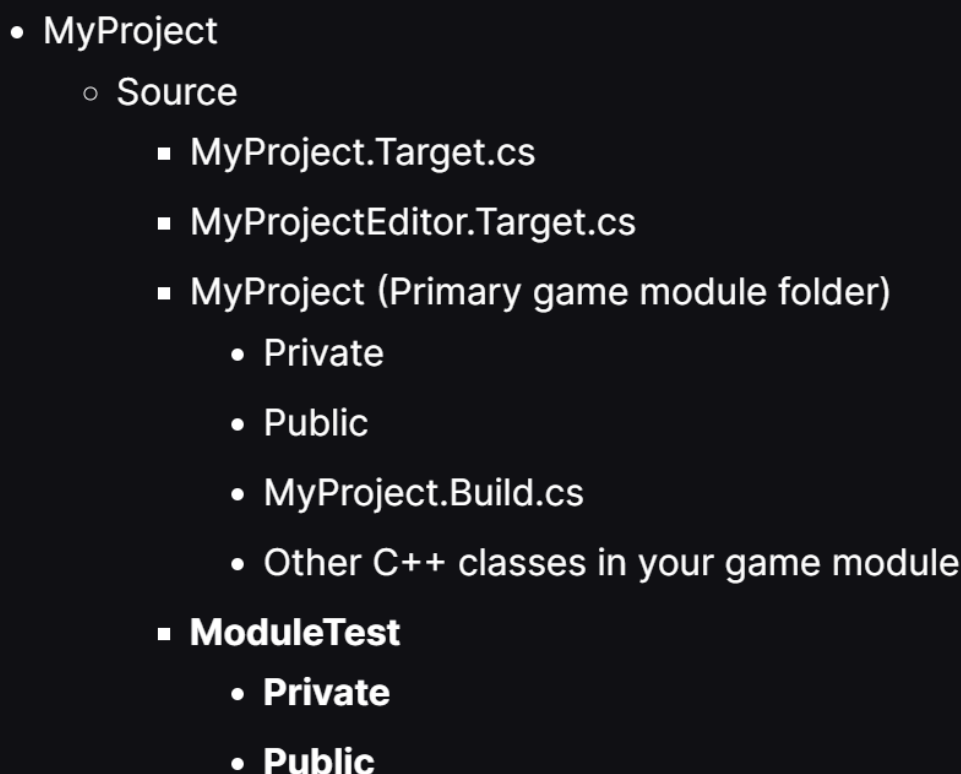
- 
- **MyProject**
 - **Source**
 - **MyProject.Target.cs**
 - **MyProjectEditor.Target.cs**
 - **MyProject (Primary game module folder)**
 - Private
 - Public
 - **MyProject.Build.cs**
 - Other C++ classes in your game module
 - **ModuleTest**
 - Private
 - Public

FIGURE 1 – Architecture type d'un projet modulaire

4.2 Build System

La création et l'intégration d'un module reposent sur le système de build d'Unreal Engine, piloté par Unreal Build Tool (UBT).

Pour ajouter un module, il est nécessaire de :

1. Créer un dossier portant le nom du module dans le répertoire **Source/**.
2. Ajouter un fichier **<ModuleName>.Build.cs** définissant les règles de compilation et les dépendances du module.
3. Déclarer le module dans le fichier **.uproject** (ou **.uplugin**) afin d'indiquer son existence et son type (Runtime, Editor, Developer, etc.).
4. Ajouter le module à la Target appropriée via le fichier **<Project>Target.cs** ou **<Project>EditorTarget.cs**, afin qu'il soit inclus dans le processus de compilation pour un contexte donné (Game, Editor, Server).

Il est important de noter qu'Unreal Engine ne compile pas les modules de manière isolée, mais compile toujours une Target. Lors de ce processus, Unreal Build Tool détecte les modules modifiés depuis la dernière compilation et ne reconstruit que ceux-ci ainsi que leurs dépendances, ce qui explique les gains de performance observés sur les temps de build.

Le système de build joue également un rôle central dans la séparation entre code Runtime et code Editor :

- Un module Runtime ne doit jamais dépendre de modules Editor.
- Les outils destinés à l'éditeur doivent être isolés dans des modules Editor dédiés.

Cette séparation garantit la compatibilité avec les builds finaux (Shipping, Server) et participe à une architecture robuste et scalable.

<https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-engine-modules>

5 Reflection System, GC Blueprint Exposure

5.1 UObject vs AActor

Les UObject sont la base dont héritent une bonne partie des classes d'Unreal, les actor en font partie, ils sont des objets dotés notamment d'un transform, pouvant être placés dans un monde.

5.2 UProperty

UPROPERTY() est une macro utilisable dans les classes dérivées d'UObject qui permet de l'intégrer au système de reflection et au Garbage Collector d'Unreal, se faisant son cycle de vie sera géré automatiquement et elle pourra apparaître dans l'éditeur.

On voit ici les différents flags et leur fonction :

Paramètre	Effet
<code>EditAnywhere</code>	La variable peut être modifiée dans l'éditeur (dans tous les contextes)
<code>BlueprintReadWrite</code>	Accessible en lecture et écriture depuis Blueprint
<code>Category="Stats"</code>	Classe la variable dans une catégorie pour l'éditeur
<code>Replicated</code>	La variable est synchronisée entre serveur et clients

Autres flags fréquents

- **EditDefaultsOnly** : modifiable uniquement sur les valeurs par défaut de la classe
- **VisibleAnywhere** : visible mais non éditable dans l'éditeur
- **Transient** : ne sera pas sauvegardé / sérialisé
- **Config** : sauvegardé dans un fichier `.ini`
- **BlueprintReadOnly** : lecture seule en Blueprint
- **ReplicatedUsing=OnRep_Function** : appelle une fonction quand la variable change côté client

5.3 TObjectPtr

Les pointeurs intelligents d'Unreal sont une version plus moderne que les pointeurs classiques, ils apportent surtout plus de sécurité étant mieux géré par le Garbage Collector.

Pointeur	Gère GC ?	Possède l'objet ?	Pour UObject ?	Usage typique
<code>TObjectPtr</code>	Oui	Non	Oui	Standard UE5, sécurité
<code>TWeakObjectPtr</code>	Non	Non	Oui	Référence faible, check <code>.IsValid()</code>
<code>TStrongObjectPtr</code>	Oui	Oui	Oui	Ownership temporaire garanti
<code>TSharedPtr</code>	Non	Oui	Non	Classes C++ non-UObject
<code>TSharedPtr</code>	Non	Oui	Non	Comme SharedPtr mais non-nullable
<code>TUniquePtr</code>	Non	Oui	Non	Ownership unique pour C++ pur

5.4 Fonctionnement du garbage collector

Le garbage collector parcourt les objets Unreal et détruit ceux qui ne sont pas marqués/référencés. D'un point de vue gameplay prog il est surtout nécessaire de comprendre ce que fait le GC plutôt que comment, afin de créer des objets et systèmes sécurisés et vaccinés aux crash.